

Triggeri (declanșatori)

- Un trigger
 - o este un bloc PL/SQL asociat unui tabel, view, scheme sau unei baze de date.
 - o trigger-ul se executa implicit ori de câte ori are loc un anumit eveniment
 - o pot fi de următoarele tipuri:
 - trigger-i la nivel de aplicație: se declanșează odată un un anumit eveniment din aplicație;
 - trigger-i la nivel de bază de date: se declanșează atunci când un eveniment asupra datelor (de ex, LMD) sau un eveniment sistem (logon, shutdown) apare asupra unei scheme sau asupra bazei de date.
- Instrucțiunea pentru crearea trigger-ilor LMD conține următoarele informații:
 - o timpul declanșării trigger-ului în raport cu evenimentul:
 - pentru tabele: BEFORE, AFTER
 - pentru view-uri nemodificabile: INSTEAD OF
 - o evenimentul declanșator: INSERT, UPDATE, DELETE
 - o numele tabelului
 - o tipul trigger-ului – precizează de câte ori se execută corpul acestuia; trigger-ul poate fi la nivel de:
 - instrucțiune (statement): corpul triggerului se execută o singură dată pentru evenimentul declanșator. Un astfel de trigger se declanșează chiar dacă nicio linie nu este afectată.
 - linie (row): corpul triggerului se declanșează o dată pentru fiecare linie afectată de către evenimentul declanșator. Un astfel de trigger nu se execută dacă evenimentul declanșator nu afectează nicio linie.
 - o clauza WHEN – precizează o condiție restrictivă
 - o corpul trigger-ului (blocul PL/SQL)
- Sintaxa comenzii de creare a unui trigger LMD este:

```
CREATE [OR REPLACE] TRIGGER [schema.]nume_trigger
    {BEFORE | AFTER}
    [INSTEAD OF]
    {DELETE | INSERT | UPDATE [OF coloana[, coloana ...]] }
    [OR {DELETE | INSERT | UPDATE [OF coloana[, coloana ...]] ...}
    ON [schema.]nume_tabel
    [REFERENCING {OLD [AS] vechi NEW [AS] nou
                  | NEW [AS] nou OLD [AS] vechi } ]
    [FOR EACH ROW]
    [WHEN (condiție) ]
    corp_trigger;
```
- Informații despre triggeri se găsesc în următoarele vizualizări ale dicționarului datelor: *USER_TRIGGERS*, *USER_TRIGGER_COL*, *ALL_TRIGGERS*, *DBA_TRIGGERS*.
- Modificarea unui declanșator constă din redenumirea, recompilarea, activarea sau dezactivarea acestuia și se realizează prin comenzi de forma:

```
ALTER TRIGGER nume_trigger ENABLE;
ALTER TRIGGER nume_trigger DISABLE;
ALTER TRIGGER nume_trigger COMPILE;
ALTER TRIGGER nume_trigger RENAME TO nume_nou;
```

Activarea și dezactivarea tuturor triggerilor asociați unui tabel se realizează prin comenzile:

```
ALTER TABLE nume_tabel  
DISABLE ALL TRIGGERS;  
ALTER TABLE nume_tabel  
ENABLE ALL TRIGGERS;
```

Eliminarea unui declanșator se face prin
`DROP TRIGGER nume_trigger;`

- Sintaxa pentru crearea unui declanșator sistem este următoarea
- ```
CREATE [OR REPLACE] TRIGGER [schema.]nume_declanșator
{BEFORE | AFTER}
{lista_evenimente_LDD | lista_evenimente_bază}
ON {DATABASE | SCHEMA}
[WHEN (condiție)]
corp_declanșator;
```

unde: *lista\_evenimente\_LDD* - CREATE, DROP, ALTER)

*lista\_evenimente\_bază* - STARTUP, SHUTDOWN, LOGON, LOGOFF, SERVERERROR, SUSPEND)

## Exerciții

1. Să se creeze un trigger care asigură ca inserarea de angajați în tabelul EMP se poate realiza numai în zilele lucrătoare, între orele 8-18.

**Obs:** Trigger-ul nu are legătură directă cu datele => este un **trigger la nivel de instrucțiune**.

2. Modificați trigger-ul anterior, astfel încât să fie generate erori cu mesaje diferite pentru inserare, actualizare, actualizarea salariului, ștergere.

3. Să se creeze un trigger care să permită ca numai salariații având codul job-ului AD\_PRES sau AD\_VP să poată câștiga mai mult de 15000.

**Obs:** Trigger-ul se declanșează de un număr de ori = nr de înregistrări inserate sau al căror câmp salary este modificat (deci are legătură cu datele din tabel) => este un **trigger la nivel de linie**.

4. Să se implementeze cu ajutorul unui declanșator constrângerea că valorile salariilor nu pot fi reduse (trei variante: *WHEN*, *IF*, *PROCEDURE*). După testare, suprimați trigger-ii creați.

5. Să se creeze un trigger care calculează comisionul unui angajat 'SA\_REP' atunci când este adăugată o linie tabelului emp sau când este modificat salariul.

**Obs:** Dacă se dorește atribuirea de valori coloanelor utilizând NEW, trebuie creați triggeri BEFORE ROW. Dacă se încearcă scrierea unui trigger AFTER ROW, atunci se va obține o eroare la compilare.

6. Să se implementeze cu ajutorul unui declanșator constrângerea că, dacă salariul minim și cel maxim al unui job s-ar modifica, orice angajat având job-ul respectiv trebuie să aibă salariul între noile limite (*RAISE* eroare; *EXCEPTION*).

7. Să se creeze un trigger check\_sal care garantează ca, ori de câte ori un angajat nou este introdus în tabelul EMPLOYEES sau atunci când este modificat salariul sau codul job-ului unui angajat, salariul se încadrează între minimul și maximul salariilor corespunzătoare job-ului respectiv. Se vor exclude angajații AD\_PRES.

Testați trigger-ul anterior:

```
UPDATE emp
```

```
SET salary = 3500
```

```
WHERE last_name= 'Stiles';
```

Ce se obține și de ce? Modificați trigger-ul astfel încât să funcționeze corect.

**Obs:** Tabelul este mutating. Pentru ca trigger-ul să funcționeze, utilizați o copie a tabelului emp în instrucțiunea SELECT din corpul trigger-ului. (aceasta este doar una dintre solutii, se vor vedea ulterior si altele).

8. a) Se presupune că în tabelul *dept* se păstrează (într-o coloană numită *total\_sal*) valoarea totală a salariilor angajaților în departamentul respectiv. Introduceți această coloană în tabel și actualizați conținutul.

b) Creați un trigger care permite reactualizarea automată a acestui câmp.

9. Să se creeze două tabele noi *new\_emp* și *new\_dept* pe baza tabelelor *employees* și *departments*. Să se creeze un view *view\_emp*, care selectează codul, numele, salariul, codul departamentului, email-ul, codul job-ului, numele departamentului și codul locației pentru fiecare angajat.

Să se creeze un **trigger de tip INSTEAD OF** care, în locul inserării unei linii direct în view, adaugă înregistrări corespunzătoare în tabelele *new\_emp* și *new\_dept*. Similar, atunci când o linie este modificată sau ștearsă prin intermediul vizualizării, liniile corespunzătoare din tabelele *new\_emp* și *new\_dept* sunt afectate.

10. Să se implementeze cu ajutorul unui declanșator restricția că într-un departament pot lucra maximum 50 de angajați.

**Obs:** Declanșatorul *TrLimitaDep* consultă chiar tabelul (*emp*) la care este asociat declanșatorul (*mutating*).

Tabelul *emp* este *mutating* doar pentru un declanșator la nivel de linie.

O soluție pentru această problemă este crearea a doi declanșatori, unul la nivel de linie și altul la nivel de instrucțiune :

- în declanșatorul la nivel de linie se înregistrează valoarea lui *:NEW.department\_id*, dar nu va fi interogată tabelul *emp*.

- interogarea va fi făcută în declanșatorul la nivel de instrucțiune și va folosi valoarea înregistrată în declanșatorul la nivel de linie.

O modalitate pentru a înregistra valoarea lui *:NEW.department\_id* este **utilizarea unui tablou indexat în interiorul unui pachet**.

```
CREATE OR REPLACE PACKAGE PdepDate AS
 TYPE t_cod IS TABLE OF dept.department_id%TYPE
 INDEX BY BINARY_INTEGER;
 v_cod_dep t_cod;
 v_NrIntrari BINARY_INTEGER := 0;
END PdepDate;
```

**Obs:** Această soluție funcționează pentru departamentele nou introduse.

Pentru testare:

- introduceți un nou departament; introduceți mai mult de 50 de angajați în departamentul inserat anterior (eventual, cu o comandă de tip INSERT INTO ... (SELECT ...));

11. Să se creeze un declanșator care:

a) dacă este eliminat un departament, va șterge toți angajații care lucrează în departamentul respectiv;

b) dacă se schimbă codul unui departament, va modifica această valoare pentru fiecare angajat care lucrează în departamentul respectiv.

**Obs:** Declanșatorul anterior realizează constrângerea de integritate *UPDATE* sau *ON DELETE CASCADE*, adică ștergerea sau modificarea cheii primare a unui tabel „părinte” se va reflecta și asupra înregistrărilor corespunzătoare din tabelul „copil”.

**Obs :** Se presupune că asupra tabelului *emp* există o constrângere de integritate:

FOREIGN KEY (department\_id) REFERENCES dept(department\_id)

În acest caz sistemul *Oracle* va afișa un mesaj de eroare prin care se precizează că tabelul *dept* este *mutating*, iar constrângerea definită mai sus nu poate fi verificată.

*ORA-04091: table MASTER.DEPT is mutating, trigger/function may not see it*

12. Să se creeze un **declanșator la nivelul bazei de date** prin care să nu se permită ștergerea informațiilor din tabelul *emp* de către utilizatorul curent. Dezactivați, iar apoi activați trigger-ul creat. Testați, iar apoi suprimați acest trigger.

13. Să se creeze un tabel care conține următoarele câmpuri: *user\_id*, *nume\_bd*, *eveniment*, *nume\_obj*, *data*. Să se creeze un **trigger sistem** (la nivel de schemă) care să introducă date în acest tabel după ce utilizatorul a folosit o comandă LDD. Testați, iar apoi suprimați trigger-ul.